

C 演習 II レポート 1

中置記法から逆ポーランド記法への変換 — 電卓プログラムの作成に向けて —

2026 年 5 月 14 日 (木)

目次

1 はじめに	1
2 基礎知識	2
2.1 中置記法と逆ポーランド記法	2
2.2 中置記法から逆ポーランド記法へ変換するアルゴリズム	2
3 課題	6
3.1 実装する機能	6
3.2 実装上の注意	8
4 補助プログラム	8
4.1 補助プログラムの入手	9
4.2 補助プログラムの使用方法	9
4.3 補助プログラムが提供する関数	9
4.4 サンプル	11
5 提出	12
5.1 提出場所と提出物	12
5.2 提出期限	14
5.3 採点の方針	15
A レポートで求める説明	16

1 はじめに

本課題ではスタックとキューに関する総合的な演習として、中置記法で記述した数式を逆ポーランド記法に変換するプログラムを作成する。これまでの全ての演習課題で培ってきた知識・技術・経験が必要になる。もし未着手の演習課題がある場合は、まずそれらに取り組んでから本課題に取り組むこと。

2 基礎知識

2.1 中置記法と逆ポーランド記法

中置記法とは、「 $3 + 2$ 」のように二項演算子を 2 つの被演算子の間に記述する書き方である。これは我々が普段よく見る数式の書き方である。一方、**逆ポーランド記法 (後置記法)** は二項演算子を 2 つの被演算子の後ろに記述する書き方である。

例えば、中置記法で記述した「 $3 + 2$ 」という式は逆ポーランド記法では「 $3\ 2\ +$ 」となる。また、中置記法の「 $(2 + 3) * 4$ 」は逆ポーランド記法では「 $(2\ 3\ +)\ 4\ *$ 」となる。これは、「 $2 + 3$ 」の部分が「 $2\ 3\ +$ 」となり、「 $() * 4$ 」の部分が「 $()\ 4\ *$ 」となるためである。なお、逆ポーランド記法では括弧は書かず、最終的には「 $2\ 3\ +\ 4\ *$ 」のように記述する。

表 1 に中置記法と逆ポーランド記法の例を示す。表 1 の式 3 と式 4 は、括弧を省略せずに表した式「 $7 + (8 * 9)$ 」と「 $(((3 + 4) + 5) + 6) * (2 + 1)$ 」を考えれば、上記の説明と同様の手順で変換できる。

表 1: 中置記法と逆ポーランド記法の例

式番号	中置記法	逆ポーランド記法
1	$3 + 2$	$3\ 2\ +$
2	$(1 + 2) * (3 + 4)$	$1\ 2\ +\ 3\ 4\ +\ *$
3	$7 + 8 * 9$	$7\ 8\ 9\ *\ +$
4	$(3 + 4 + 5 - 6) * (2 + 1)$	$3\ 4\ +\ 5\ +\ 6\ -\ 2\ 1\ +\ *$

上述の変換手順は、中置記法で記述した数式の解釈方法を人間が知っているからこそ行える手順である。一方、コンピュータから見ると中置記法で記述された数式は単なる文字の羅列に過ぎず、コンピュータ自身は数式の解釈方法を知らない。そのため、コンピュータに上述の手順を行わせることは単純にはできない。次節では、中置記法から逆ポーランド記法へ変換するための全く別の方法について説明する。

2.2 中置記法から逆ポーランド記法へ変換するアルゴリズム

中置記法で記述した数式を逆ポーランド記法に機械的に変換する方法の 1 つとして、スタックとキューを用いるものがある。本節ではまず用語について説明し、その後、アルゴリズムについて説明する。

2.2.1 用語

数式の構成要素を**トークン**と呼ぶ。トークンは数式を構成する最小の単位であり、演算子や括弧、数値が 1 つのトークンとなる。例えば、「 $10 * (20 + 3.14)$ 」という数式を構

成するトークンは、10, *, (, 20, +, 3.14,) である。

今回説明するアルゴリズムでは、演算子や括弧を表すトークンを一時的に退避するためにスタックを用い、逆ポーランド記法に変換したトークンの列を格納するためにキューを用いる。以降、このスタックを**演算子スタック**、キューを**数式キュー**と呼ぶ。

本課題では、「キーボードから数式を入力してトークンに分割する処理」は4節で説明する補助プログラムとして提供する。この補助プログラムではトークンを文字列(char*型)として表現している。そのため、演算子スタックや数式キューは文字列を格納できなければならない。**文字列を格納するスタックやキューは、第3,4回の演習課題で取り組んでいることを思い出して欲しい。**

2.2.2 アルゴリズム

中置記法の数式を逆ポーランド記法に変換するアルゴリズムを図1と表2に示す。図1の最初の繰り返し処理が逆ポーランド記法へ変換するための主たる処理となる。ここで、「中置記法の式のトークンを先頭から順に…」というのは、10*(20+3.14)という式の場合、10, *, (, 20, +, 3.14,)の順に調べるということである。

繰り返し処理の内部において、今調べているトークンを t と表記する。 t と演算子スタックの状態に応じて、表2の処理を行う。 t が数字である場合は演算子スタックの状態に依らず t を数式キューにenqueueすることになる。

t が演算子である場合はやや複雑な動作となる。特に、演算子スタックの頂上が演算子(s と表記する)である場合が複雑である。表2の動作では、「 t と s のどちらが先に適用する演算子か」を判定する必要がある。**この判定は第1回演習のレポート関連課題で取り組んでいること思い出して欲しい。**

また、「演算子スタックの頂上が t よりも後に適用すべき演算子になるまで演算子スタックをpopして…その後、 t を演算子スタックにpushする」という部分は、要約すると「 t よりも先に適用すべき演算子を全てpopしてから t をpushする」ということである(popしたものは全てenqueueする)。**この動作は第3回演習のレポート関連課題で取り組んだ「自分よりも大きいデータを追い出してからpushする」と本質的に同等である。**これらのレポート関連課題を上手くアレンジしてプログラムを考えると良い。

繰り返しを終了した後、演算子スタックに残っている記号をpopして数式キューにenqueueしていけば、最終的に数式キューに格納されているトークンの並びが逆ポーランド記法の式になっている。なお、このときに演算子スタックから「(」をpopした場合、入力した中置記法の式は括弧の対応が取れていないことになり、エラーである。

図2に、「1-4*3+2」という式に対してこのアルゴリズムを適用したときの動作例を示す。この例をよく見てアルゴリズムの動作をよく把握してほしい。

- 演算子スタックと数式キューを初期化する(空にする)。
- 中置記法の式の構成要素(トークン)を先頭から順に注目しながら以下を繰り返す。
 - 今注目中のトークン t と演算子スタックの状態に応じて、表2の通り処理する。
- 演算子スタックが空になるまで以下を繰り返す。
 - $a \leftarrow$ 演算子スタックからpopした記号。
 - a が"("であればエラー。そうでなければ数式キューにenqueueする。
- 数式キューに格納されているトークンを先頭から順に出力する(この時点で数式キューには逆ポーランド記法で表現した数式のトークンが並んでいる)

図 1: 中置記法から逆ポーランド記法への変換するアルゴリズムの概略

表 2: 注目中のトークン t と演算子スタックの状態による動作

t	演算子スタックの状態		
	空	頂上の記号	
		"("	演算子 s
数字			t を数式キューにenqueueする。
演算子	t を演算子スタックにpushする。		t が s よりも先に適用すべき演算子である場合、 t を演算子スタックにpushする。そうでない(s が先に適用すべき演算子である)場合、演算子スタックが空になるか、演算子スタックの頂上が"("になるか、演算子スタックの頂上が t よりも後に適用すべき演算子になるまで演算子スタックをpopして取り出した演算子を数式キューにenqueueする。その後、 t を演算子スタックにpushする。
"("			t を演算子スタックにpushする。
")"	エラー		演算子スタックから"("をpopするまで、popして取り出した演算子を数式キューにenqueueする。取り出した"("はenqueueしない。なお、"("をpopするまでに演算子スタックが空になった場合はエラーである。

注目中のトークン	動作	動作後の状況
(注目前)		先頭← 数式キュー →末尾 底← 演算子スタック →頂上 紙面の都合上、スタックは横に倒して表現する
1 - 4 * 3 + 2	演算子スタックと数式キューを空にしておく。	
$\downarrow t$ 1 - 4 * 3 + 2	t は数字であるため、 t を数式キューにenqueueする。	1
$\downarrow t$ 1 - 4 * 3 + 2	t は演算子であり、演算子スタックは空であるため t を演算子スタックにpushする。	1 -
$\downarrow t$ 1 - 4 * 3 + 2	t は数字であるため、数式キューにenqueueする。	1 4 -
$\downarrow t$ 1 - 4 * 3 + 2	t は演算子であり、演算子スタックの頂上の“-”よりも先に適用すべき演算子であるため、 t を演算子スタックにpushする。	1 4 - *
$\downarrow t$ 1 - 4 * 3 + 2	t は数字であるため、数式キューにenqueueする。	1 4 3 - *
$\downarrow t$ 1 - 4 * 3 + 2	演算子スタックが空になるか、頂上が“(”になるか、 t が頂上よりも先に適用すべき演算子になるまで、頂上の演算子をpopして数式キューにenqueueする。その後 t をpushする。	1 4 3 * - + スタックが空になったためtをpushした。 +よりも-を先に適用することに注意。
$\downarrow t$ 1 - 4 * 3 + 2	t は数字であるため数式キューにenqueue	1 4 3 * - 2 +
1 - 4 * 3 + 2	演算子スタックに残っている演算子をpopしながら数式キューにenqueueする。最終的に数式キューの内容が逆ポーランド記法で表した式となっている。	1 4 3 * - 2 +

図 2: アルゴリズムの動作例

3 課題

中置記法で記述した数式を入力し、その数式を逆ポーランド記法に変換して出力するプログラムを作成しなさい。

図3に実行例を示す。「enter>」で始まる行でキーボードから中置記法の式を入力している。この入力を行うプログラムは4節に示す補助プログラムで提供する。入力した数式を図1と表2に示した手順で逆ポーランド記法の式に変換し、その結果を出力する。この変換と出力を行うプログラムが、各自が作成する部分である。

```
% ./main.exe
enter> 5 + 2 * 3  ←キーボードから中置記法の式を入力.
5 2 3 * +        ←逆ポーランド記法に変換した式を出力しプログラムを終了.

% ./main.exe
enter> 2 ^ 2 ^ 3
2 2 3 ^ ^

% ./main.exe
enter> ( ( 1 + 2 ) * 3 + 4 ) * 5
1 2 + 3 * 4 + 5 *

% ./main.exe
enter> 10 + 5 - 3 - 2 + 1
10 5 + 3 - 2 - 1 +

% ./main.exe
enter> 3 + 1 - 4 + 1 + 5 * 9 * 2 / 6 * 5 / 3 - 5
3 1 + 4 - 1 + 5 9 * 2 * 6 / 5 * 3 / + 5 -

% ./main.exe
enter> 10 + 20 * ( 30 - 40 + 50 ) - ( 60 + 70 )
10 20 30 40 - 50 + * + 60 70 + -

% ./main.exe
enter> 1+(5*(8+2)-1*(1+(9+2)*1)+(1-8))-5
1 5 8 2 + * 1 1 9 2 + 1 * + * - 1 8 - + + 5 -
```

図 3: 実行例

3.1 実装する機能

本課題で実装する機能の一覧を以下に示す。

- 必須項目 (A1)

括弧を含まず、加算と乗算のみから成る数式について中置記法から逆ポーランド記法に変換できる。

- 必須項目 (A2)

べき乗を表す二項演算子 $\wedge$ を含む数式も変換できる。ここで、 $a \wedge b$ は a の b 乗 (a^b) を表す。べき乗演算子は**右結合**¹であることに注意すること。すなわち、 $2 \wedge 2 \wedge 3$ は $2 \wedge (2 \wedge 3) = 256$ を表す。また、べき乗演算子は加算・減算・乗算・除算の演算子よりも優先度は高い。すなわち、 $4 * 2 \wedge 2 \wedge 3 / 8$ は $4 * (2 \wedge (2 \wedge 3)) / 8 = 128$ を表す。なお、C 言語では $\wedge$ はべき乗演算子ではないことに注意すること。

- 加点点項目 (B)

括弧を含む数式も変換できる。

- 加点点項目 (C)

減算と除算を含む数式も変換できる。加減算演算子と乗除算演算子は**左結合**¹であることに注意すること。すなわち、同じ優先度の演算子を括弧を省略して書いた場合、左側の演算子を先に適用する。例えば、中置記法の「 $4+3-2+1$ 」という式は「 $((4+3)-2)+1$ 」という意味であり、逆ポーランド記法では「 $4\ 3\ +\ 2\ -\ 1\ +$ 」となる。

- 加点点項目 (D)

変換した逆ポーランド記法の数式を計算し、その計算結果を出力できる。ここで、数式は小数点以下も含めて計算し、計算結果は printf 関数の %f の形式で出力すること。実現した演算子の種類が多いと加点点も多くなる。なお、**逆ポーランド記法の数式を計算するアルゴリズムは第3回演習のレポート関連課題で取り組んでいる**。その課題では逆ポーランド記法の式はプログラム引数として入力したが、今回は数式キューに格納されている点が異なる。また、計算に用いるスタックの実現方法は、例えば以下のような方法が考えられる。どのように実現しても構わない。

- 演算子スタックとは別に計算途中の実数値を格納するためのスタックを用意する。この方法を採用した場合、3.2 節の第2項目に注意すること。
- 演算子スタックを流用する。数式を正しく変換できた場合、変換後は演算子スタックは空状態になっているため、これを計算用に流用する。ただし、演算子スタックは文字列を格納するスタックであるため、計算途中の実数値は文字列として扱う必要がある。後述の補助プログラムにおいて実数値を文字列に変換する ftoa 関数を用意しているので、必要であれば用いてもよい。

C 言語で a の b 乗を計算するには、数学ライブラリの pow 関数を用いる。pow 関数は b が実数である場合も計算できる。数学ライブラリを使用するには、プログ

ラムの先頭部分に `#include <math.h>` を追加し、gcc コマンドでコンパイルする際に `-lm` を指定する (C 演習 I(4 学科) の教科書の 10.3 節も参照)。

- 加点点項目 (E)

スタックに関するプログラムを stack.c と stack.h に、キューに関するプログラムを queue.c および queue.h に記述し、**適切に分割コンパイルしている**。さらに、**その分割が適切であることの根拠**をレポートできちんと説明できている。

レポートが受理されるためには少なくとも必須項目 (A1)(A2) を実現しなければならない。加点点項目 (B)–(E) を実現した場合はさらなる加点点の対象となる。いずれの項目も単にプログラムを実装するだけでなく、レポート中でわかりやすく適切に説明する必要がある (C 演習 I と同様)。なお、(A1)(A2)–(D) まで実現すると、本課題の副題にある「電卓プログラムの作成」が実現できたと言える。

3.2 実装上の注意

本課題で実装するプログラムについて、以下に示す点に注意すること。これらが守られていない場合や求められている説明が不十分である場合は再提出もしくは低評価となる。

1. 数式をキーボードから入力する処理は、補助プログラムに含まれる enter 関数 (4.3 節) を用いること。enter 関数以外でキーボード入力を行ってはいけない。
2. 原則として教科書第 1,2 章に基づいたスタックとキューの実装を用いること。そうでない実装²を用いる場合、教科書の実装との違い、および、あえてその実装を用いることの意図・利点・採用理由などもレポートで十分に説明すること。
3. スタックやキューの長さ (STACK_SIZE や QUEUE_SIZE) は、少なくとも 100 以上に設定すること。なお、1 回の入力で与えられるトークンの個数は 100 以下であることを想定してよい。

4 補助プログラム

本レポート課題のプログラムに必要な数式の入力や数式の構成要素への分割といった処理は、補助プログラムとして用意している。以降では、補助プログラムの入手方法および使用方法について説明する。

¹左結合と右結合については第 1 回演習のレポート関連課題で取り組んでいる。

²変数の役割や名前の違いも含む。加点点項目 (D) に取り組んだ場合にも注意すること。

注意 1 %はプロンプトと呼ばれる目印であり、コマンドは%より右側の部分です。
注意 2 #から行末までは補足説明であり、コマンドや出力結果ではありません。

```
% cd ~/kadai/padv26/           #C 演習 II の課題ディレクトリに移動

#report1 関連のファイルをまとめた圧縮ファイルをダウンロード
% wget http://150.89.235.70/padv26/lec06/files/report1.tgz
...(出力省略).....'report1.tgz'へ保存完了 [...]
    #もし wget でダウンロードできない場合は、wget の代わりに curl -O を使う

% tar xzpf report1.tgz          #ダウンロードしたファイルを解凍
                                #何も表示されなければ成功。report1 ディレクトリができる。
% cd report1                    #report1 ディレクトリに移動
% pwd                           #****/kadai/padv26/report1 に居ることを確認
% rm -f ../report1.tgz          #確認できたらダウンロードしたファイルを消しておく

% ls -l                          #ファイルの確認
-rw-r--r-- 1 xxx xxx 0 4月 28 17:41 compile.txt      #提出物ベース
-r--r--r-- 1 xxx xxx 6818 4月 28 17:41 lib1.c        #補助プログラム
-r--r--r-- 1 xxx xxx 1791 4月 28 17:41 lib1.h        #補助プログラム
-rw-r--r-- 1 xxx xxx 0 4月 28 17:41 main.c          #提出物ベース
-rw-r--r-- 1 xxx xxx 0 4月 28 17:41 report.txt       #提出物ベース
-rw-r--r-- 1 xxx xxx 0 4月 28 17:41 result.txt      #提出物ベース
```

図 4: report1 関連ファイルのダウンロードとその確認の手順

4.1 補助プログラムの入手

まず、図 4 に記載しているコマンドを実行し、レポート 1 に関連するファイルを入手する。図 4 の最後の `ls -l` コマンドで表示される `lib1.c` と `lib1.h` が補助プログラムであり、その他のファイルは提出物のベースとなるファイルである。

4.2 補助プログラムの使用方法

補助プログラムの使用のイメージを図 5 に示す。本課題で各自が作成するプログラムは `main.c` であり、補助プログラムは `lib1.c` と `lib1.h` である。図 5 に示すように、`main.c` に `#include "lib1.h"` と記述し、かつ、`gcc` コマンドで `lib1.c` と一緒にコンパイルすることで、補助プログラムが提供する関数を使用することが可能となる。

補助プログラム (`lib1.c`) が提供する関数の機能について以降に示す。

4.3 補助プログラムが提供する関数

- `int enter(char *token[], int n)`

`main.c` (自分で作成するプログラム)

```
#include <stdio.h>
#include "lib1.h"

int main(void){
    ...
}
```

`lib1.c` で定義された補助プログラムを使用するために必要。

`% gcc -o main.exe main.c lib1.c`

コンパイル

2つのプログラムを同時にgccに渡す

`lib1.c` (ダウンロードした補助プログラム)

```
...
...
(編集せずにそのまま使用する)
...
...
```

実行プログラム
(main.exe)

実行

`% ./main.exe`

図 5: `lib1.c` のコンパイル方法

キーボードから数式を読み込み、数式の構成要素（トークン）に分割する。分割した各トークンは文字列の形で、第 1 引数に指定した配列 `token` に格納される。第 2 引数 `n` には配列 `token` の要素数を指定する。分割したトークンの数が `n` よりも多い場合はエラーとなり、プログラムは強制終了される。なお、トークンが 1 つも含まれない場合（例えば、エンターキーだけをを入力した場合）、入力処理を繰り返す。戻り値は分割したトークンの個数である。すなわち、戻り値を変数 `c` に格納した場合、`token[0]` から `token[c-1]` にトークンが格納されていることになる。特別な状況として、キーボードから `q` または `Q` のみを入力した場合、`-1` を返す。これは入力を終了することを判断するためのものである。

- `int isNumber(char *str)`

引数に与えた文字列が数値を表す文字列であれば 1 を返し、そうでなければ 0 を返す。ここで、数値を表す文字列とは、`0, 1, 2, ..., 9` および小数点 (`.`) のみから成る文字列である。符号 (`+`, `-`) で始まる文字列は該当しない。また、小数点が 2 回以上現れる文字列や先頭または末尾に現れる文字列も該当しない。

- `int isDouble(char *str)`

`isNumber` 関数が 1 を返すような文字列に加えて、符号 (`+`, `-`) で始まる数字列も 1 を返す。

- `int isOperator(char *str)`

引数に与えた文字列が演算子を表す文字列であれば 1 を返し、そうでなければ 0 を返す。ここで、演算子を表す文字列とは、`"+", "-", "*", "/", "%", "^"` である。

"%"については3.1節で述べた項目には含まれていないが、剰余演算子などを追加するという工夫に用いてもよい³。

- char *ftoa(double x)

引数に与えた double 型の値を文字列に変換し、その文字列を返す。この関数は1回呼び出す度にメモリ領域を消費する。少なくとも1000回は呼び出せるが、それ以上呼び出した場合、メモリ領域を使い切り次第、エラーメッセージを出力してプログラムを強制終了する。本課題に取り組むにあたり、この関数がメモリ領域を使い切る状況は想定しなくてもよい。

4.4 サンプル

補助プログラムが提供する関数を用いたプログラムの例を以下に示す。このプログラムを書き写し、動作を確認しなさい。

```
sample.c: サンプルプログラム

#include <stdio.h>
#include "lib1.h"      ←補助プログラムを使用するために必要
#define MAX_TOKEN_NUM 100 ←1回の入力で分割できるトークンの最大数

int main(void){
    char *token[MAX_TOKEN_NUM]; ←分割したトークンを格納する配列
    int tnum, i;

    tnum = enter(token, MAX_TOKEN_NUM); /* 最初の入力 */
    while (tnum > 0){

        /* トークンを先頭から1つずつ表示する */
        for (i=0; i<tnum; i++){
            printf("token[%d]:%-4s  ", i, token[i]);
            printf("isNumber:%d  ", isNumber(token[i]));
            printf("isOperator:%d\n", isOperator(token[i]));
        }

        tnum = enter(token, MAX_TOKEN_NUM); /* 次の入力 */
    }
    return 0;
}
```

このプログラムを sample.c とする。以下にコンパイルと実行結果を示す。

```
% gcc sample.c lib1.c -o sample.exe -Wall ← sample.c と lib1.c を書く

% ./sample.exe
enter> 10+3.14*(3-1) ←キーボード入力
token[0]:10      isNumber:1  isOperator:0 ←最初の"10"は数値
token[1]:+       isNumber:0  isOperator:1 ←次の"+"は演算子
token[2]:3.14    isNumber:1  isOperator:0 ←次の"3.14"は数値
token[3]:*       isNumber:0  isOperator:1 ←次の"*"は演算子
token[4]:(       isNumber:0  isOperator:0 ←次の"("はどちらでもない
token[5]:3       isNumber:1  isOperator:0
token[6]:-       isNumber:0  isOperator:1
token[7]:1       isNumber:1  isOperator:0
token[8]:)       isNumber:0  isOperator:0

enter> q ←次の入力。「q」を入力すると戻り値-1を返してwhile文を抜ける。
```

このサンプルプログラムの while 文の内部に、図1のアルゴリズムに相当するプログラムを記述していくことになると思われる⁴。

なお、サンプルプログラムでは while 文を用いて入力を繰り返している。一方、図3の実行例では1回の入力と出力でプログラムを終了している。本課題で作成するプログラムでは、数式の入力を1回に限るか続行するかは自由に決めてよい。ただし、入力を続行する場合、2回目以降の入力でもきちんと変換できることをよく確認すること⁵。

5 提出

5.1 提出場所と提出物

提出場所は ~/kadai/padv26/report1 である。ここに所定のファイルを配置することで提出したことになる。c1-byod を使用している場合は putkadai コマンドで提出することを忘れないこと。提出物は以降に示すファイルである。

5.1.1 main.c, stack.c, stack.h, queue.c, queue.h

main.c は main 関数を含むソースファイルである。加点項目 (E) に取り組んだ場合、スタックやキューに関するファイルも提出ディレクトリに配置する。ファイル名の綴りを間違えないように注意すること。必要であればこれら以外にもファイルを分割しても良いが、その場合は後述の compile.txt でコンパイルコマンドを適切に記述すること。

³この場合、剰余演算子の優先順位や結合性はC言語のものを参考に自分で考えること。また、そのことをレポート中で説明すること。

⁴当然、main 関数に全てのプログラムを書くことと読み難くなるので適度に関数に分割するべきである。

⁵1回目の処理の残骸が残っているため2回目以降の動作が失敗するというミスはありがちである。

5.1.2 compile.txt

work59,5a と同様に、作成したプログラムをコンパイルして main.exe という名前の実行プログラムを生成するコマンドを記述する。端末で「sh compile.txt」と実行して main.exe が生成できればよい。ただし、main.exe の実行はしてはいけない。

このファイルは教員が提出物を動作確認する際に用いる。「sh compile.txt」で main.exe を生成できない場合や main.exe の実行まで行なっている場合は原則として再提出となる。

5.1.3 result.txt

作成したプログラムの実行結果を書く。**実装項目ごとに**数式の入力部分（「enter>」で始まる行）と実行結果をそのままの形（コピー&ペースト）で載せること。

result.txt には実行結果（数式の入力部分とその実行結果）のみを載せる。**実行結果に対する説明・解釈・考察などは result.txt には書かず、report.txt に書くこと**。また、report.txt の説明中に実行結果を参照しやすいように、各実行結果には識別子（番号やラベル）を付けること。これらが守られていない場合、減点対象となる。

5.1.4 report.txt

以下の形式に沿ってまとめること。

- 完成させた項目
- プログラムの説明
- 動作確認
- 考察、工夫点、その他

まず先頭部分で完成させた項目（3.1 節）を列挙する。この箇所には、取り組んだけど完成できなかった項目は書かない。そのような項目は最後のその他に記述する。その後、プログラムの説明、動作確認の方法、考察、工夫点などを書く。

プログラムの説明は**要点をまとめてわかりやすく**書くこと。また、「出来上がったプログラムに対する説明」ではなく、これからプログラムを作成していくという段階において、どのような方針やアイデアで作成していくか、何のために（何を狙って）どのような機能を実現しようとするのか、機能の実現にあたって考慮しなければならない問題点は何か、その問題点に対してどのようにプログラムを実現するべきか、といったことを書くこと。付録にこれらの記述例を示すので参考にすること（2024,2025 年度 C 演習 I のレポート 1 において示したものと同等である）。

必須項目・加点項目・工夫点などに関して上記のような説明が無い、あるいは不十分であると判断された場合、その項目に対する加点は無い。

単にプログラムを日本語に直訳しただけのような文章⁶はプログラムの説明とはみなされず、極めて低い評価となる。**なぜならば、そのような文章はプログラムを全く理解しておらず、他人のプログラムをコピーしただけでも書いてしまうためである**。レポート課題についてしっかりと理解したことをアピールするための説明であると認識してほしい。

動作確認については、result.txt に記載した実行結果を参照しながら⁷、**その結果で完成できていると判断した根拠を文章で説明する**。単に「実行結果より正しいことがわかる」といった説明だけでは、読者には全くわからない。このような説明は正しさの判断を読者に丸投げしていることになり、レポートとしては低い評価となる。**なぜ正しいと判断できるか**（例えば別の手段で式を計算して正しい値になることを示すなど）を、読者にわかるように説明すること。なお、**原則として実行結果を report.txt に載せてはいけない。これらは result.txt に載せること**。そうならない場合は再提出、または著しく低評価となる。

考察や工夫がある場合は文章で説明すること。プログラム上では確かに工夫が施されていても文章での説明がない場合は採点において工夫点には数えられない。これら以外にも、全体的に適度に章や節、段落に分けるなどして読みやすい構成にすることを心がけること。読みやすさも加点の対象となる。

5.2 提出期限

提出期限は **6 月 3 日 (水) の朝 9:00** である。この時刻にプログラムを用いて自動的に提出物を回収する。提出ディレクトリや各ファイルの名前が 1 文字でも違っていると回収できず、未提出として扱われるので注意すること。軽微な手違いやネットワークトラブルなどの場合でも期限までに提出できていないものは提出遅れとして扱う。不測の事態に備えて 1 日以上前に提出を終えておくことが望ましい。

出題から提出期限まで約 3 週間ある。**特に最初の 1 週間は C 演習 II の他の課題は無い**ため、**レポート課題に時間をかけやすい期間である**（2 週目以降は次回の演習課題と同時に並行して取り組むことになる）。この最初の 1 週間にできる限りプログラムを完成に近付けておくことが大事である。report.txt の作成にかかる時間も多めに見積もっておくことが無難である。以上を踏まえて計画に取り組んでほしい⁸。

⁶ 「～を代入し、次に～を代入し、～が～である間繰り返して…」というように、プログラムを読み上げているだけの文章を指す。このような文章はむしろプログラムを理解できていないことをアピールしており、採点者の心証は悪い。

⁷ どの実行結果を参照しているかわかるように説明すること。

⁸ 各作業にかかる時間を見積もり、計画的に作業を進めることはソフトウェア開発者に求められる素養でもある。

5.3 採点の方針

受理とは、プログラムが動作するなどの最低限の要件を満たし、評価の対象になったことを表す。最低限のレポートで受理された場合、レポートの得点はそれほど高くはないことに注意すること⁹。

合格水準に達するためには、単に動くプログラムを作るだけでなく、report.txt をしっかり書き、期限内に提出することが必要である。他にも多くの項目を実現したり、考察をしたり、工夫点を盛り込むなど、積極的に取り組んでほしい。

以下に採点の基本的な方針を示す。この方針に留意して取り組むこと。

1. レポート課題は成績を判定するための重要な材料である。そのため、他人のレポートやプログラムを写したり他人に写させるなどの行為は不正行為と判断し、本レポート課題は受理しないか、あるいは大減点とする。
 - 友達にほんの少しだけ見せた際にコピーされたという事例があるが、その場合、見せた方も不正行為と判断される。レポートやプログラムを他人に見せたり見られたりしないよう厳重に取り扱いに気をつけること。
 - コピーしたものに小手先の細工を施してコピーでないように偽装することも不正行為である。
 - 質問サイトやSNSなどでプログラムやレポートの提示を求めたり写したりする行為や生成 AI の出力を成果物として提出する行為も不正行為として扱う。
2. 提出期限に遅れた場合や再提出の判定となった場合は減点となる。また、最終提出期限（後日連絡）を過ぎた後の提出物は受理されない。完成度の高いレポートを提出期限内に提出すること。
3. 以下に示す項目に1つでも合致する場合は受理せず、再提出となる。
 - (a) ファイルが無かったり名前のスペルミス等により、教員が回収できない。
 - (b) `sh compile.txt` を実行したときに `main.exe` を生成できない、あるいは実行が終わらない。
 - (c) 必須項目 (A1)(A2) が実現できていない。
 - (d) report.txt の内容が乏しい。500 字未満である場合は即再提出となる。
 - (e) その他、完成度が著しく低いなど。
4. 抽象化の観点から望ましくないプログラムを書いている場合、減点もしくは再提出となる。例えば、利用者側のプログラムからデータ構造の内部変数を直接使っている場合などが該当する。

5. しっかりと書かれた考察や工夫点は加点の対象となる。

- (a) 積極的に加点項目や独自の機能を実装してほしい。ただし、単に実装しただけは加点にはならず、report.txt で読者にとってわかりやすく説明できていることが加点の要件である。5.1.4 節の内容をよく確認すること。
- (b) わかりやすい文章になっている場合も加点の対象となる。
- (c) 工夫を行ったとしても、report.txt で工夫の説明と効果の両方を書いていないければ加点は無い。読者がわかるように書いていることが加点の条件である。

6. report.txt の作成にあたり、5.1.4 節を熟読すること。指定の形式になっていなかったり、内容に不備がある場合は低評価となる。例えば、動作確認の書き方や正しさの根拠の説明が不十分である場合、評価はかなり低くなる。
7. report.txt の説明が不明瞭である場合、口頭試問を行うことがある。また、**提出されたソースプログラムの理解度を確認するために口頭試問を行うこともある。**
8. 繰り返しになるが、最低限の内容で受理されても評価は低い。この場合、他のレポートやテストで挽回できなければ、全てのレポートが受理されていても不合格になる。場合によってはテストが満点でも挽回しきれないこともあり得る。report.txt をしっかりと書き、加点項目に取り組んだり工夫や考察をするなどして、積極的に高得点を目指してほしい。
9. 本課題で作成したプログラムやレポートを SNS などインターネット上に公開することは禁止する。公開した場合、不正行為と判断する。また、C 演習 II の授業終了後（卒業後も含む）に公開がわかった場合、配属ゼミの指導教員等に相談する。

付録

A レポートで求める説明

レポート (report.txt) で書くプログラムの説明は、完成版のプログラムに到るまでの経緯・考慮点・注意点・方針などを説明する形で書くこと。**単に出来上がったプログラムを解説しているだけでは評価は低い。**以下に、(C 演習 (4 学科) の) 教科書の work49 を題材として、レポートで求められている書き方とそうでない書き方の例を示す。

A.1 C 演習 I(4 学科) の work49 の問題文

キーボードから座標 (x, y) を入力します。入力された点と図 4.5 に示される図形の位置関係を判定して、実行結果のように表示させるプログラムを作成しなさい。図中の正方形の一辺の長さは 10 とします。また円は原点を中心とし正方形に外接しています。ただし、線上は内側に位置するものとし、判定基準は次の通りとします。

⁹ レポートの得点が 5 割未満になることもあり得る。2 つのレポートが受理され、第 14 回に実施するテストが満点であっても、レポート点が低いために不合格になることもある（過去に実際にある）。

1. 正方形と外接円の両方の内側
2. 正方形の外側で外接円の内側
3. 正方形と外接円の両方の外側

A.2 作成したプログラム

```
#include <stdio.h>
int main(void){
    double x, y;
    int flag1=0, flag2=0;

    printf("x 座標を入力して下さい：");
    scanf("%lf", &x);
    printf("y 座標を入力して下さい：");
    scanf("%lf", &y);

    if (-5<=x && x<=5 && -5<=y && y<=5){
        flag1 = 1;
    }
    if (x*x + y*y <= 5*5 + 5*5){
        flag2 = 1;
    }

    if (flag1 == 1 && flag2 == 1){
        printf("点 (%.1f,%.1f) は、正方形と外接円の両方の内側です。\\n", x, y);
    }else if (flag1==0 && flag2==0){
        printf("点 (%.1f,%.1f) は、正方形と外接円の両方の外側です。\\n", x, y);
    }else {
        printf("点 (%.1f,%.1f) は、正方形の外側で外接円の内側です。\\n", x, y);
    }
    return 0;
}
```

A.3 report.txt の例

以降に2つの説明の例を示す。その後、それらの違いについて説明する。

【説明 A】 レポートで求められていない説明（低評価となる説明）

このプログラムでは、まず点 (x, y) の座標を入力する。その後、 $-5 \leq x \leq 5$ かつ $-5 \leq y \leq 5$ であることを判定し、この条件が成り立てば flag1 を 1 に設定している。flag1 は宣言時に 0 としているため、条件が成り立たなければ 0 のままである。

次に、 $x*x + y*y \leq 5*5 + 5*5$ であることを判定し、この条件が成り立てば flag2

を 1 に設定している。flag2 も宣言時に 0 としているため、条件が成り立たなければ 0 のままである。

最後に、flag1, flag2 の内容に基づいて、正方形の内外および外接円の内外について判定している。

【説明 B】 レポートで求められている説明

まず、点 (x, y) が正方形に含まれるか否かを判断する方法について考える。教科書の問題文より、今回扱う正方形は原点を中心とし、一辺の長さが 10 である。したがって、正方形の x 座標の範囲は、原点からそれぞれ正と負の方向に 5（一辺の半分）で挟まれる区間となる。同様に、正方形の y 座標の範囲も、原点を中心に正と負の方向に 5 で挟まれる区間となる。したがって、点 (x, y) が正方形に含まれるか否かの判定は、線上も内側であることに注意すると、 $-5 \leq x, y \leq 5$ であるか否かを判定すればよい。この判定は C 言語では「 $(-5 \leq x \ \&\& \ x \leq 5) \ \&\& \ (-5 \leq y \ \&\& \ y \leq 5)$ 」と表現すればよい。

次に、点 (x, y) が正方形の外接円に含まれるか否かを判断する方法について考える。この外接円は原点を中心とするため、点 (x, y) の原点からの距離が外接円の半径以下であれば、外接円に含まれると判定できる。点 (x, y) の原点からの距離は $\sqrt{x*x + y*y}$ である。外接円の半径 r は、正方形の頂点と円が接していることから、原点から正方形の頂点までの距離と等しい。正方形の右上の頂点の座標は、前述の「正方形に含まれるか否かの判定」の議論より、 $(5, 5)$ である。したがって、 $r = \sqrt{5*5 + 5*5} = \sqrt{50}$ である。以上より、点 (x, y) が外接円に含まれるか否かは、「 $\sqrt{x*x + y*y} \leq \sqrt{50}$ 」であるか否かを調べればよい。

なお、現時点（C 演習 I の work49 出題時点）において、C 言語で平方根を求める方法はまだ授業では扱われていないため、どのように「 $\sqrt{x*x + y*y} \leq \sqrt{50}$ 」を判定するプログラムを記述するかが問題になる。ここで、 $0 \leq x*x + y*y$ であることを考慮すると、2 乗したものを比較、すなわち、「 $x*x + y*y \leq 50$ 」としても同じ結果が得られるため、プログラム上ではそのように記述することにする。⁹

以上を踏まえ、点 (x, y) が入力されたあと、正方形に含まれるか否かの判定結果と外接円に含まれるか否かの判定結果をそれぞれ変数（フラグ）に格納し、それぞれのフラグを調べて「正方形の外接円の両方の内側」、「正方形の外側で外接円の内側」、「正方形と外接円の両方の外側」を判定し、その結果を出力すればよい。

説明 A と説明 B の大きな違いは、A は完成したプログラムについて後から解説している書き方になっているのに対し、B はプログラムをこれから構築していくための方

⁹今回は 2 乗したものを比較するという方法をとったが、他にも work68 や worka1（共に C 演習 I）を活用する方法が考えられる。いずれの場合でもそれらを用いることにした経緯なども説明すること。

針・考慮点・注意点などを説明した書き方になっている点（完成品に対する解説ではない点）である。

Aのような説明は課題の解き方を理解していなくても書けてしまう。したがって、課題に対する理解度を読み取ることができないため、評価は低くなる（たとえ実際には課題を理解していたとしても）。Bのように、どのような点に注意し、どのようなプログラムにすればよいと考えたか、といったことを重点的に説明すること。

なお、今回の説明Bでは工夫点や考察の説明は無いが、工夫や考察を行った場合は以下のように書くとよい。

1. 説明Bのように、工夫に関する経緯・考慮点・注意点など書く。
2. 「プログラムの説明」を終えたあとに、「工夫点」という節を作り、行った工夫の一覧を示す。
3. 「考察」という節を作り、作成したプログラムや結果に対して考察を行う。

上記の2や3が無く、1の説明の中に含めてしまうと、工夫点が説明に埋もれてしまい、工夫点として認識されないことがあるので、「プログラムの説明」の節とは別に「工夫点」の節を作って工夫点についてアピールすると良い。考察についても同様である。